Requerimientos de servidores — ambiente de desarrollo (PREP 2027)

Este documento es para Infraestructura. Describe los **dos servidores** que necesitamos para el ambiente de **desarrollo** (no Azure/productivo todavía). Una vez funcionando, este mismo diseño se usará como guía para dimensionar los servidores productivos, así que cualquier decisión de nomenclatura/puertos/dominio que se tome aquí conviene documentarla para replicarla después.

	Servidor 1 — PREP (Docker)	**Servidor 2 — Extractor + Bot de difusores**
Contiene	API NestJS + frontend Angular, en un contenedor Docker	Dos servicios Node.js/PM2: el Extractor (lee la BD MSSQL "PREP de Rugerio" y publica cortes) y el Bot de difusores (rastrea el estado de los cortes en los difusores)
SO preferido	Linux (RHEL 9 si está disponible)	Windows Server (2019/2022)
Acceso a internet	Solo saliente, para pulls/parches del SO (el contenedor no necesita internet en runtime)	Saliente obligatorio (push HTTPS de cortes hacia el Servidor 1/difusores, y llamadas del bot a la API de Telegram)
Rutas públicas	La que se defina con Infraestructura (ver sección 4)	`/extractor-prep/` y `/bot-difusores/` sobre el mismo host
RAM mínima	8 GB	8 GB
Disco mínimo	50 GB SSD	50 GB SSD
Quién instala qué	Ver sección 3	Ver sección 3

Ya construimos y probamos la imagen Docker localmente (`deployment-prep-2027/`), así que las características de abajo no son un estimado teórico: son las que usamos en las pruebas reales.

---

1. Servidor 1 — despliegue del contenedor Docker (PREP)

1.1 Sistema operativo

Preferencia: RHEL 9 (x86-64/amd64), por ser lo que maneja normalmente la infraestructura del IECM. Si no hay licenciamiento/imagen de RHEL disponible para desarrollo, estas alternativas Linux sirven igual (probadas o compatibles con lo probado):

Opción	**Estado**	**Notas**
RHEL 9	Preferida	SELinux en modo `enforcing` (no desactivar — ver 1.5)

Opción	Estado	Notas
Ubuntu Server 22.04/24.04 LTS	Probada	Requisito mínimo original del documento de arquitectura
Debian 12/13 estable	Probada (imagen base del contenedor es Debian 13 “trixie”)	AppArmor activo por defecto

Importante: la imagen Docker es siempre `linux/amd64`. **No sirve un servidor ARM** (Ampere, AWS Graviton, etc.) — necesitaríamos publicar una imagen multi-arquitectura aparte, que hoy no existe. Windows con Docker Desktop/WSL2 tampoco es una opción válida para este servidor (ver nota al final de esta sección) — el servidor Docker de Linux es el que pedimos aquí.

1.2 Especificaciones de hardware (ambiente de desarrollo)

Recurso	Mínimo	Recomendado
CPU	2 vCPU	4 vCPU
RAM	**8 GB**	16 GB
Disco	**50 GB SSD**	100 GB SSD (SO + imagen Docker + datos de cortes SQLite + logs)
Red	1 NIC con salida a internet (para `apt` / `dnf` / `docker pull` de parches)	—

El contenedor en sí está limitado a 2 CPU / 1 GB RAM (`deploy.resources.limits` en `compose.external.yaml`) — el mínimo de 8 GB de RAM y 50 GB en SSD de la tabla es el piso que pedimos para el servidor completo (SO, Apache/Nginx, motor Docker y margen de crecimiento), no lo que consume el contenedor por sí solo. Se prioriza SSD sobre HDD por la escritura/lectura frecuente de SQLite en cada corte.

1.3 Software a instalar en el servidor (fuera del contenedor)

- **Docker Engine + Docker Compose plugin** (no Docker Desktop — este es un servidor Linux headless). Repositorio oficial de Docker para la distro correspondiente (`download.docker.com/linux/<distro>`).
- **Apache HTTP Server (`httpd` en RHEL, `apache2` en Ubuntu/Debian) o Nginx**, como proxy inverso con TLS. Ya tenemos un ejemplo de configuración probado para Apache (`deployment-prep-2027/apache/prep.conf.example`) — si el estándar de Infraestructura es Nginx, podemos adaptarlo (misma idea: terminar TLS ahí, reenviar a `127.0.0.1:3000`).
- Certificado TLS para el dominio que se asigne (Let’s Encrypt o el CA interno del IECM).
- Nada más — el runtime de Node.js/NestJS/Angular vive **dentro** de la imagen Docker; el host no necesita Node, pnpm, npm ni ninguna dependencia de la aplicación instalada directamente.

1.4 Puertos

Puerto	Dirección	Uso
443/TCP	Entrante, público	HTTPS — único puerto que debe quedar expuesto a los usuarios finales, atendido por Apache/Nginx
80/TCP	Entrante, público	Solo para redirigir HTTP→HTTPS (no servir contenido)
3000/TCP	Interno, `127.0.0.1` únicamente	El contenedor NestJS escucha aquí; **no debe exponerse fuera del host** — el compose ya lo publica solo en loopback (`127.0.0.1:3000:3000`)
22/TCP (SSH)	Entrante, restringido	Solo administración, idealmente desde VPN/IPs de oficina, no público
2375/2376/TCP	—	**Deben quedar bloqueados/cerrados** — son el socket remoto de Docker sin TLS, no los usamos

El contenedor corre como usuario no-root (`node` , uid 1000), con `cap_drop: ALL` y `no-new-privileges` , y el filesystem de la imagen es de solo lectura salvo dos carpetas de datos — ninguno de estos son ajustes que Infraestructura deba hacer, ya vienen así en la imagen.

1.5 Otros requisitos del SO

- **Firewall**: `ufw` (Ubuntu/Debian) o `firewalld` (RHEL) — permitir 443 y 80, bloquear 2375/2376.
- **SELinux (RHEL)**: dejarlo en `enforcing` . El volumen de datos (`/opt/prep/data`) necesita el contexto `container_file_t` (`semanage fcontext` + `restorecon`) — si no se hace, la app falla con `Permission denied` al escribir, aunque los permisos Unix estén bien. Esto lo puede aplicar Infraestructura al crear la carpeta, o lo hacemos nosotros si nos dan el acceso `sudo` puntual.
- **AppArmor (Debian)**: no requiere ajustes adicionales, es compatible con la configuración del contenedor tal cual.
- Carpetas de datos en el host, por ejemplo `/opt/prep/{data,logs,keys}` — deben persistir entre despliegues (ahí vive la base SQLite de resultados activa y su archivo histórico). Si hay política de backup de disco por convención de Infraestructura, esta carpeta debería incluirse.

---

2. Servidor 2 — Extractor + Bot de difusores (Windows Server, con salida a internet)

Este servidor aloja **dos servicios Node.js independientes**, cada uno bajo su propia ruta:

Servicio	Repo	Ruta pública	Puerto interno	Función
Extractor	`extractor/backend-extractor-prep-2026` (`microservice-exportacion`)	`/extractor-prep/`	3004	Lee la BD MSSQL "SCD" (captura de votos en vivo), la espeja a SQLite local, y publica los "cortes" resultantes por HTTPS hacia el Servidor 1 y otros difusores
Bot de difusores	`bot-difusores-prep`	`/bot-difusores/`	3001	Rastrea el estado de los cortes publicados en cada difusor y notifica por Telegram

Ninguno de los dos está contenerizado hoy (son procesos Node.js/Express corriendo directo sobre el SO), y ambos comparten el mismo servidor porque son ligeros y están relacionados operativamente (el bot vigila lo que el Extractor publica). Por eso este servidor necesita conectividad tanto a la red interna (SCD/MSSQL) como salida a internet (push de cortes + API de Telegram).

2.1 Sistema operativo

Windows Server 2019 o 2022 (x86-64), según lo pedido. No hay nada en el código que requiera Windows específicamente (es Node.js puro, `mssql` / `tedious` funciona igual en Linux), pero se pide así porque: - El equipo que administra la base MSSQL "SCD" de origen normalmente opera en infraestructura Windows/Active Directory. - Facilita el uso de herramientas de administración de SQL Server (SSMS, drivers ODBC) si Infraestructura las necesita para diagnóstico de la conexión al SCD.

Si por disponibilidad se prefiriera Linux para este servidor también es viable (mismo runtime Node.js), pero mantenemos Windows Server como la opción solicitada.

2.2 Especificaciones de hardware (ambiente de desarrollo)

Recurso	Mínimo	Recomendado
CPU	2 vCPU	4 vCPU
RAM	**8 GB**	16 GB
Disco	**50 GB SSD**	100 GB SSD (SO + espejo SQLite del SCD + bitácora histórica de cortes)
Red	1 NIC con salida a internet (HTTPS saliente) + ruta de red hacia el servidor MSSQL "SCD"	—

Aunque cada servicio individualmente es ligero, el mínimo de 8 GB / 50 GB en SSD es el mismo piso que pedimos para el Servidor 1, para que ambos ambientes de desarrollo queden dimensionados igual y esa base sirva de referencia directa al pasar a producción.

2.3 Software a instalar

- **Node.js 22.x LTS**, instalado vía **nvm** — en Windows Server, la herramienta equivalente es **nvm-windows** (no el `nvm` de bash/macOS, que no corre en Windows). Instalar nvm-windows y luego `nvm install 22` / `nvm use 22`, en vez del instalador MSI directo de nodejs.org — así queda igual de fácil actualizar de LTS en el futuro sin reinstalar todo. Ambos servicios (`extractor-prep` y `bot-difusores`) corren sobre la misma instalación de Node 22.
- **Git** (el script `start:dev` del Extractor hace `git pull` antes de arrancar — a evaluar si en Windows Server se usa ese flujo o si nosotros desplegamos el código directamente sin ese `git pull` automático; lo definimos con el equipo de desarrollo antes del primer despliegue).
- **PM2** como gestor de procesos para ambos servicios (`pm2 start index.js --name extractor-prep`, `pm2 start index.js --name bot-difusores`), más **`pm2-windows-startup`** para que PM2 y los procesos que administra arranquen solos tras un reinicio del servidor — es la única herramienta de gestión de procesos que pedimos aquí (se descarta NSSM para no mantener dos mecanismos distintos). La puede instalar el equipo de desarrollo si nos dan acceso administrador puntual, o Infraestructura si prefiere mantenerlo como parte de su estándar de servicios.
- **IIS con el módulo Application Request Routing (ARR) + URL Rewrite**, para terminar TLS y hacer proxy inverso **por ruta** hacia los dos procesos Node (equivalente al Apache/Nginx del Servidor 1): `/extractor-prep/*` → `127.0.0.1:3004`, `/bot-difusores/*` → `127.0.0.1:3001`. Con esta configuración ningún puerto de aplicación queda expuesto directamente, solo 443 vía IIS.
- Certificado TLS para el dominio/IP que se asigne a este servidor.

2.4 Puertos

Puerto	Dirección	Uso
3004/TCP (configurable por `PORT` en `.env`)	Interno, `127.0.0.1` únicamente	Extractor — IIS lo enruta desde `/extractor-prep/`
3001/TCP (configurable por `PORT` en `.env`)	Interno, `127.0.0.1` únicamente	Bot de difusores — IIS lo enruta desde `/bot-difusores/`
443/TCP	Entrante, público	HTTPS único punto de entrada — IIS/ARR distribuye a `/extractor-prep/` y `/bot-difusores/` según la ruta
80/TCP	Entrante, público	Solo para redirigir HTTP→HTTPS
1433/TCP	Saliente, hacia el servidor MSSQL del SCD (confirmar IP real del SCD de desarrollo con Infraestructura)	Conexión de lectura del Extractor a la base origen

Puerto	Dirección	Uso
443/TCP	Saliente, hacia el Servidor 1 (PREP), hacia cualquier otro difusor configurado, y hacia `api.telegram.org`	Envío de cada corte SQLite (Extractor) y notificaciones del Bot
3389/TCP (RDP)	Entrante, restringido	Solo administración, idealmente desde VPN/IPs de oficina

2.5 Otros requisitos

- **Windows Firewall**: reglas explícitas de entrada solo para los puertos de la tabla anterior; todo lo demás cerrado por defecto. 3004 y 3001 no deberían tener regla de entrada abierta a la red si quedan detrás de IIS en loopback.

---

3. División de responsabilidades (Infraestructura vs. Desarrollo)

Tarea	Responsable
Aprovisionar el servidor (VM, SO base, parches del SO, dominio/IP, DNS)	**Infraestructura**
Instalar Docker Engine + Compose plugin (Servidor 1)	**Infraestructura**
Instalar nvm-windows + Node.js 22 LTS, PM2 + `pm2-windows-startup`, IIS/ARR (Servidor 2)	**Infraestructura**
Instalar Apache/Nginx y configurar el vhost/proxy con TLS	**Infraestructura**, con la plantilla que ya tenemos (`apache/prep.conf.example`)
Configurar firewall/reglas de red (puertos, bloqueo 2375/2376, reglas RDP/SSH)	**Infraestructura**
Emitir/renovar certificados TLS	**Infraestructura**
Cargar la imagen Docker (`docker load`) y levantar el stack (`docker compose up`)	**Desarrollo**, con los scripts que ya existen (`scripts/install-release.sh`, `compose.external.yaml`)
Configurar `.env` de cada servicio (secretos de aplicación)	**Desarrollo**
Desplegar/actualizar el código del Extractor y del Bot de difusores en el Servidor 2, y administrarlos vía PM2	**Desarrollo**
Configurar las reglas de IIS/ARR para `/extractor-prep/` y `/bot-difusores/`	**Infraestructura**, con Desarrollo indicando los puertos internos (3004 y 3001)
Ajustes de SELinux/contexto de volumen si aplica (RHEL)	**Infraestructura**, con el comando ya documentado, o Desarrollo si nos dan acceso puntual

En resumen: pedimos que Infraestructura entregue el servidor con el SO, Docker/Node.js y el proxy (Apache/Nginx/IIS) instalados y los puertos abiertos según las tablas de arriba; a partir de

ahí, Desarrollo se encarga de cargar/actualizar la aplicación con los scripts que ya construimos y probamos.

---

4. Dominio/URL de acceso — puntos a resolver con Infraestructura

Para el ambiente de desarrollo, usar un dominio es opcional — acceder por IP (con o sin ruta, p. ej. `145.0.40.3/sistema-1/`) es aceptable mientras se valida el esquema. Sin embargo, sí debería considerarse y decidirse desde ahora pensando en producción: el mismo par de servidores (RHEL/Linux para el contenedor Docker, Windows Server para el Extractor y el Bot de difusores) se va a replicar tal cual quede aquí, y un dominio real en producción (certificado, DNS, CORS) es prácticamente obligatorio de cara al uso interno — no algo que convenga dejar para último momento. Por eso, aunque en desarrollo no se asigne dominio todavía, conviene dejar resueltos con Infraestructura los puntos de abajo (sub-ruta vs. subdominio, quién resuelve el nombre, tipo de certificado) como si sí lo fuéramos a usar, para que el mismo diseño de proxy/CORS/certificado se traslade a producción sin tener que rehacerlo:

En otros servidores de desarrollo se alojan varios sistemas sobre la misma IP, diferenciados por ruta, por ejemplo `145.0.40.3/sistema-1/`. Ese no es el caso del Servidor 1: ahí **solo se desplegará el contenedor de PREP**, un único sistema, sin otros servicios compartiendo el mismo host — por lo que no hay necesidad de distinguir por ruta como en esos servidores compartidos. Aun así, conviene confirmar con Infraestructura lo siguiente, porque afecta cómo configuramos el proxy y el propio frontend Angular:

1. **Acceso administrativo (SSH/RDP)**

Referencias internas (ya construidas y probadas)

- `deployment-prep-2027/docs/INSTALACION-POR-SISTEMA-OPERATIVO.md` — comandos exactos de instalación de Docker por distribución Linux (Ubuntu/RHEL/Debian), firewall y notas de SELinux/AppArmor.
- `deployment-prep-2027/docs/OPERACION-DIFUSOR-EXTERNO.md` — runbook completo de instalación/rollback del contenedor en un servidor externo.
- `deployment-prep-2027/apache/prep.conf.example` — plantilla de proxy inverso Apache con TLS, probada.
- `deployment-prep-2027/compose.external.yaml` — definición exacta de puertos/volúmenes/límites del contenedor en un servidor de despliegue.
- `extractor/backend-extractor-prep-2026/CLAUDE.md` — arquitectura del microservicio Extractor.